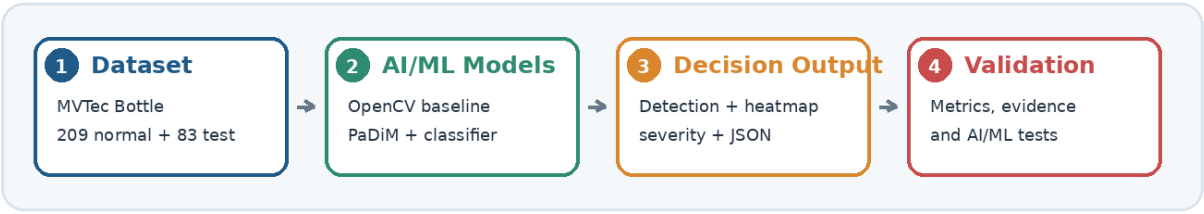
VisionInspect AI

AI/ML Module Implementation Report

Manufacturing Defect Detection, Explainability and Quality Decision Support



Submission field	Details
Project	VisionInspect AI
Assigned module	AI/ML module
Prepared by	Himanshu Sekhar Parhi
Mentor	Sai Jyoteesh
Programme	Infosys Springboard Programme
Submission date	10.07.2026

Report purpose

This report documents the AI/ML module of VisionInspect AI from dataset understanding and image preparation through anomaly detection, defect classification, localization, severity scoring and structured output. It distinguishes the deployed OpenCV baseline from the trained PaDiM candidate and links technical claims to notebook, source-code, artifact and test evidence.

Image processing

Validation, RGB handling, resizing, enhancement and defect cues.

Anomaly intelligence

OpenCV baseline and trained PaDiM anomaly detection/localization.

Decision intelligence

Defect classification, severity scoring and QA action mapping.

Platform readiness

Saved artifacts, backend-ready JSON, frontend evidence and Pytest validation.

Table of Contents

Navigation note

Page numbers are verified after final pagination. Figure and section numbering are kept consistent for traceability.

List of Figures and Evidence Index

1. Executive Summary

VisionInspect AI is a manufacturing visual-inspection platform that converts bottle images into traceable quality evidence. The assigned AI/ML module validates images, prepares model-specific inputs, detects visual abnormality, localizes suspected defect regions, classifies defect type when an anomaly is present, calculates severity and returns a structured inspection result for platform consumption.

The implementation combines an interpretable OpenCV reference-difference baseline with a trained PaDiM anomaly detector. The OpenCV path is configured as the deployed demonstration/fallback detector, while PaDiM is the stronger trained candidate for image-level detection and pixel-level localization. When the active detector marks an image as defective, a frozen ResNet18 feature extractor with StandardScaler and Logistic Regression estimates the defect category and confidence.

Executive implementation outcome

The AI/ML module extends beyond notebook experiments: reusable Python modules, saved model artifacts, explainability outputs, structured result fields, quantitative evaluation and recorded software-validation evidence are included.

Capability	Implemented result	Evidence
Input handling	Readable image is validated before AI/ML inference	Image metadata and validation screenshots
Detection and localization	Baseline and PaDiM produce abnormality evidence	Difference map, anomaly map, mask and heatmap
Defect classification	Classifier returns category and confidence	Confusion matrix, class metrics and sample predictions
QA decision	Rule-based severity maps evidence to action	Severity table and decision-gate flow
Integration	JSON-compatible fields and heatmap are exposed at the integration boundary	JSON output and result/heatmap pages
Reliability	Model metrics and recorded software tests are kept as separate assurance layers	Metrics and Pytest output

Key results at a glance

Component	Reported result	Interpretation
OpenCV baseline	Accuracy 51.81%; recall 36.51%; ROC-AUC 92.06%	Useful fallback/demo, but selected threshold misses defects
PaDiM	Image AUROC 99.84%; Image F1 98.39%	Strong image-level anomaly detection
PaDiM localization	Pixel AUROC 98.21%; Pixel F1 71.36%	Strong anomaly ranking; moderate exact overlap
Defect classifier	Verified 83-sample accuracy 92.77%; error 7.23%	Strong category prediction; separate 93.18% record is retained only as historical until provenance is supplied
Software validation	Submission evidence: 19 tests passed in 31.93 seconds	Validated tested software behaviour and output consistency

2. Project Context, Objectives and Scope

2.1 Manufacturing inspection problem

Manual visual inspection is repetitive, time-consuming and sensitive to operator attention, fatigue and experience. Product appearance can also vary with lighting, background, camera angle and orientation. These factors make consistent defect detection difficult to scale and make inspection decisions harder to audit later.

2.2 Role of the AI/ML module

The AI/ML module converts a raw product image into measurable evidence. Detection answers whether an abnormality is present; localization identifies where it is; classification identifies the likely defect category only after an anomaly is detected; severity estimates operational seriousness; and QA logic maps the result to Pass, Review or Fail. Backend and frontend components are treated only as consumers of the AI/ML output.

2.3 Objectives

Reliable image handling Reject unreadable or unsupported inputs before model inference.	**Anomaly evidence** Produce a good/defective signal, anomaly score and localization map.
Defect identification Return class label and probability/confidence.	**Actionable decision** Convert technical evidence into severity and QA action.
Reusable integration Expose predictable fields to backend and frontend modules.	**Measured reliability** Separate model evaluation from software testing and deployment validation.

2.4 Scope boundary

Included in this report	Outside the current validated scope
MVTec Bottle dataset, validation, preprocessing, baseline, PaDiM, classifier, localization, heatmaps, severity, artifacts, AI/ML output contract and module tests	Factory camera hardware, authentication, database internals, frontend implementation and full platform acceptance testing
Integration evidence limited to showing that the platform can consume the AI/ML result	Claim of production certification before threshold calibration and real-environment validation

3. Current Implementation Status

The report distinguishes what is currently deployed, what is trained and evaluated, and what still requires production activation. This prevents strong offline metrics from being incorrectly interpreted as the performance of the currently deployed Render path.

Component	Current status	Runtime purpose	Readiness note
OpenCV baseline	Configured as the deployed Render demonstration/fallback path	Lightweight good/defective fallback/demo	Evaluation metrics are tied to the 99th-percentile threshold, not automatically to the runtime threshold
PaDiM checkpoint	Trained and evaluated; checkpoint available; deployment disabled in the supplied Render configuration	Image-level anomaly detection and pixel localization	Preferred production-oriented candidate; deployment activation pending

Component	Current status	Runtime purpose	Readiness note
Defect classifier	Trained artifact available	Predicts category and confidence	Verified 83-sample result is 92.77%; artifact provenance should remain versioned
Severity engine	Implemented rule-based logic	Maps evidence to Low/Medium/High/Critical and QA action	Requires policy validation with domain stakeholders
Backend/frontend integration	Demonstrated at the AI/ML output boundary	Consumes image input and returns structured fields plus heatmap evidence	Needs live model version alignment in deployed environment
Automated tests	Recorded submission evidence: 19 tests passed	Validates selected software behaviours	Does not replace predictive or factory validation

Production interpretation

The currently deployed baseline should be presented as an explainable demonstration/fallback. The trained PaDiM results represent the stronger model candidate, not the present Render detector until its checkpoint and inference path are activated in deployment.

Achieved versus pending

Achieved

Complete data-to-output implementation, model artifacts, evaluation evidence, frontend display and automated tests.

Pending

PaDiM activation on Render, threshold calibration, real-camera validation, false-negative monitoring and operational governance.

4. Development and Runtime Workflows

4.1 Development lifecycle

The development lifecycle explains how the module was created and validated. It begins with dataset understanding and image-processing experiments, then progresses through baseline development, PaDiM training, classifier evaluation, artifact packaging and integration testing.

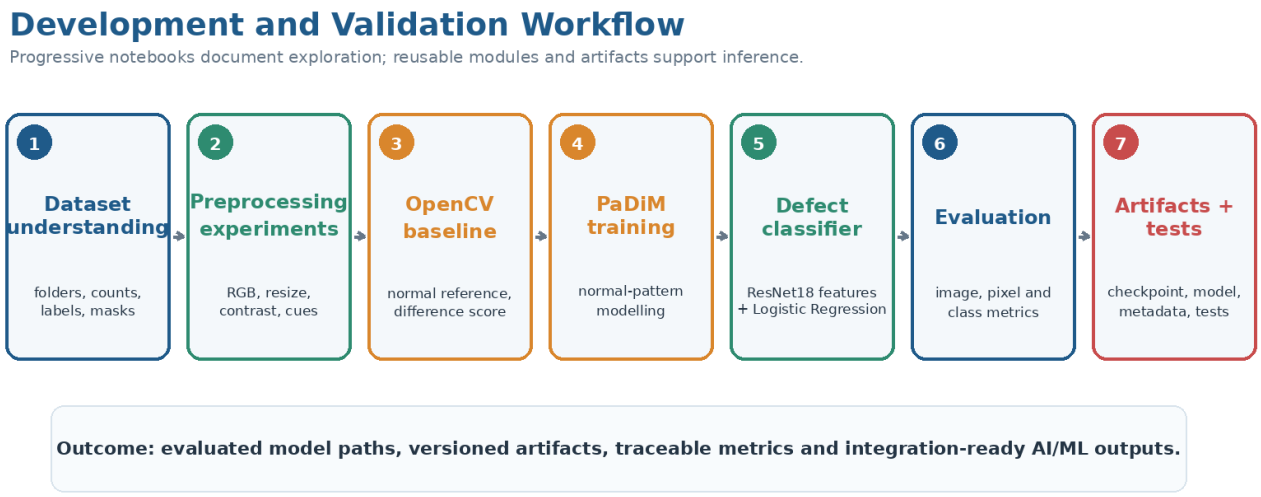


Figure 1. Development workflow from dataset understanding to saved artifacts and tested integration.

What this proves: The project follows a progressive engineering lifecycle rather than moving directly from dataset to deployment.
Observation: Notebooks preserve exploration and evaluation evidence; reusable modules and artifacts support inference.

4.2 Runtime inspection lifecycle

At runtime, the uploaded image is decoded, validated and converted into the representation required by the active detector. The configured detector is either the OpenCV baseline or PaDiM. If no anomaly is detected, the module returns a Good result without running defect classification. If an anomaly is detected, localization evidence is retained and the ResNet18-based classifier estimates defect type and confidence before severity and QA decision logic are applied.

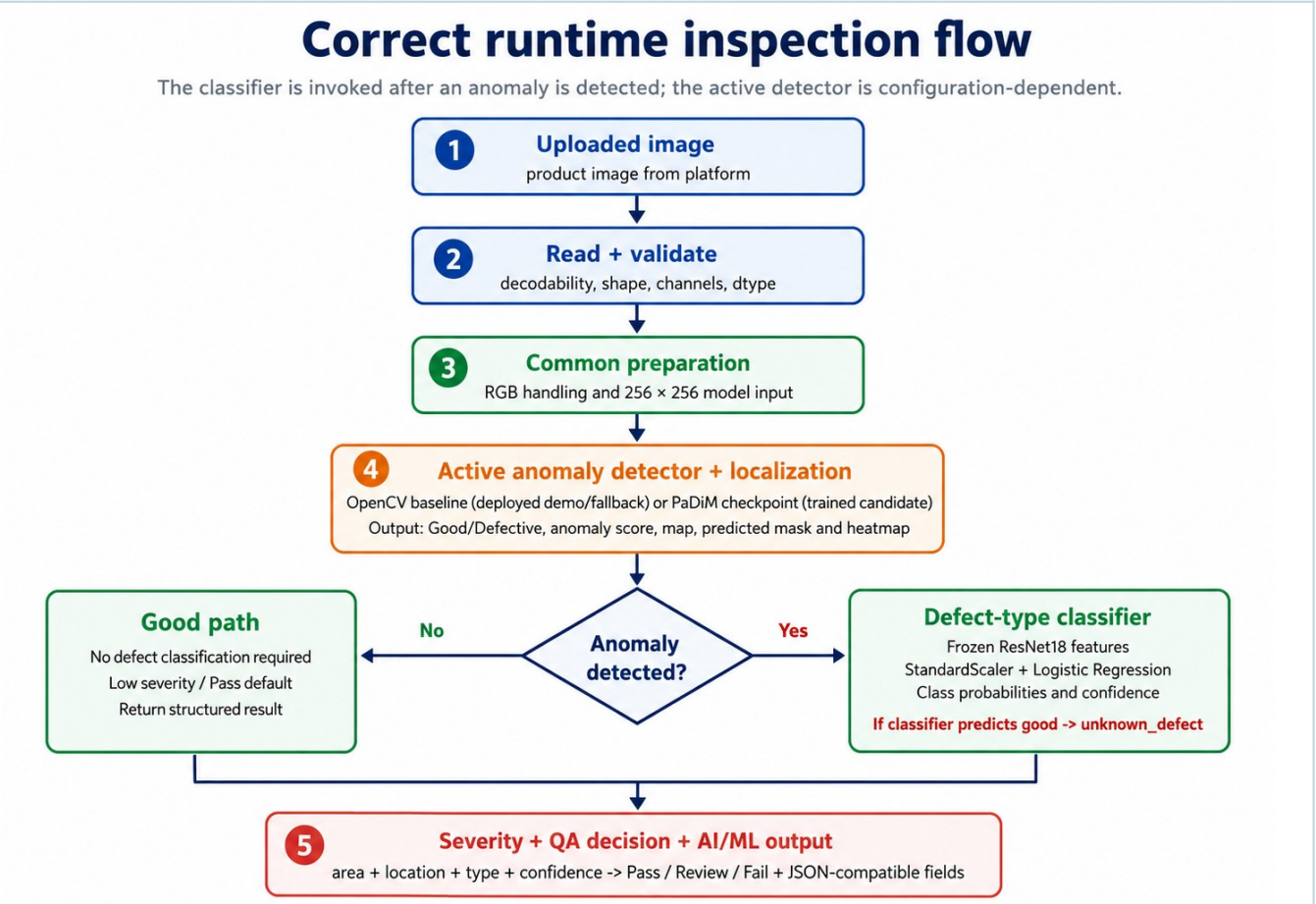


Figure 2. Correct runtime inspection flow with detector selection and conditional defect classification.

What this proves: The anomaly detector runs first; classification is invoked only for images marked defective.

Observation: A Good result bypasses defect classification, while a detected anomaly proceeds through classification, severity and structured output.

Limitation: The active anomaly engine depends on configuration: the supplied Render configuration disables PaDiM and uses the baseline fallback path.

5. Technology Stack and Method Selection

Technology choices were matched to the role of each stage rather than using one model for every task. Image-processing tools support validation and explainable baselines; anomaly detection learns normal appearance; the classifier predicts defect category; and testing tools validate software behaviour.

Python + NumPy Core language and numerical image-array operations.	**OpenCV** Image reading, BGR/RGB conversion, baseline difference maps, thresholds and edge cues.
Pandas + Matplotlib Dataset summaries, metrics tables and reproducible visual evidence.	**PyTorch + ResNet18** Deep visual feature extraction for classification and anomaly-model support.
Anomalib / PaDiM Normal-pattern anomaly detection with image and pixel-level outputs.	**scikit-learn** StandardScaler, Logistic Regression and classification metrics.
FastAPI-compatible output Structured dictionary/JSON fields for platform integration.	**Pytest** Automated checks for dataset, preprocessing, prediction, artifact and severity behaviour.

Why each method was selected

Method	Question answered	Selection reason
OpenCV baseline	Is the test image visibly different from a normal reference?	Fast, interpretable and suitable for demonstration/fallback use
PaDiM	Does the image contain a deviation from learned normal appearance, and where?	Normal-only training fits industrial anomaly detection and provides localization
ResNet18 + Logistic Regression	What defect class is visually most likely?	Strong pretrained features with lightweight probabilistic classification
Heatmap and mask	Where is the abnormal evidence?	Makes the decision visually inspectable
Rule-based severity	What QA action should follow?	Combines model evidence with transparent policy weights

6. Architecture and Component Responsibilities

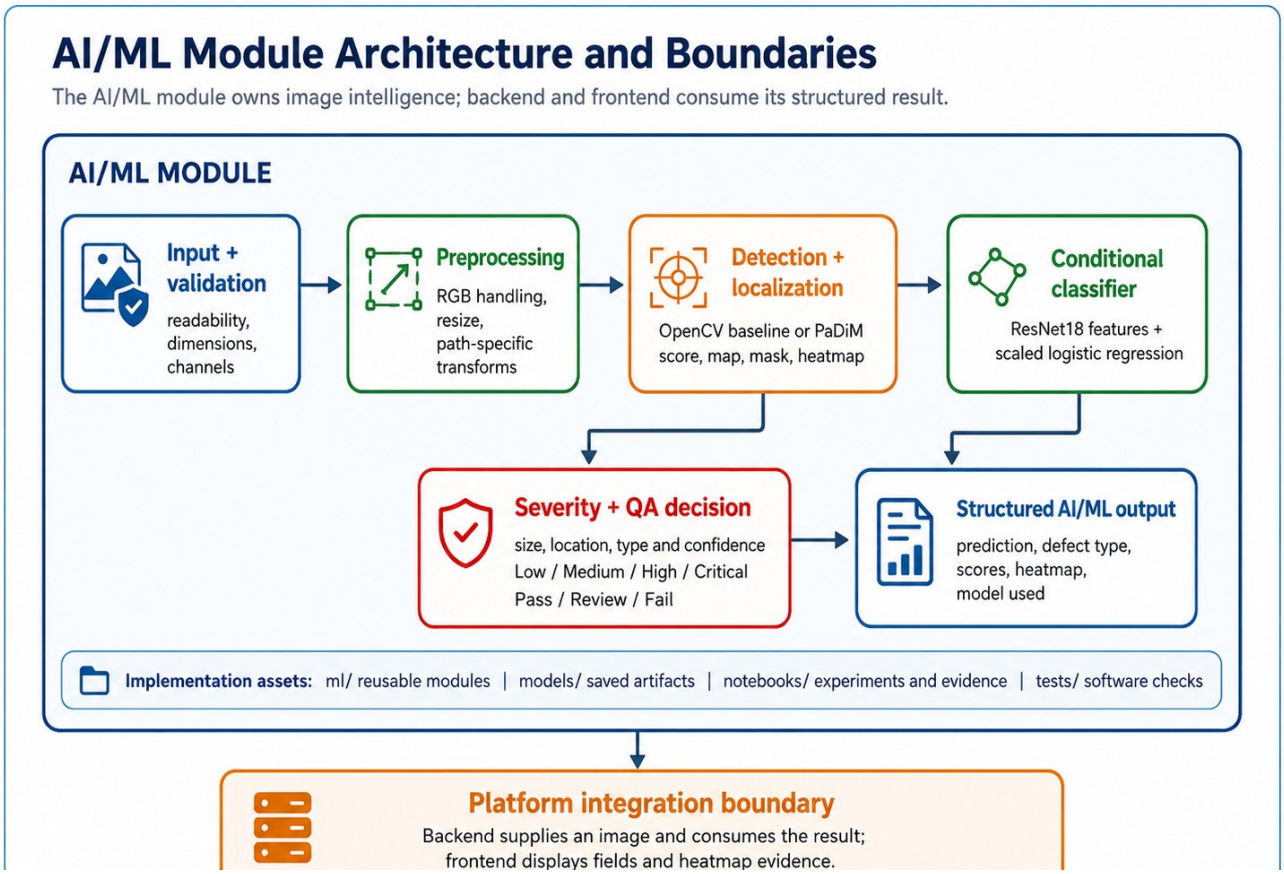


Figure 3. AI/ML architecture showing module boundaries, conditional classification and structured output.

What this proves: The diagram separates AI/ML ownership from backend/frontend consumption and mirrors the implemented decision sequence.

Observation: Detector/localizer evidence feeds the classifier only for detected anomalies; severity merges area, location, type and confidence.

6.1 What each component does

Component	Input	Core responsibility	Output
`image_basics.py`	Image path or array	Inspect shape, channels, dtype, pixels and representations	Image metadata and visual understanding evidence
`dataset_loader.py`	Dataset root	Load split, class, image and ground-truth paths	Validated sample records
`preprocessing.py`	Validated image	Apply shared and baseline-oriented preprocessing	Model-ready image and visual cues
`baseline_detector.py`	Preprocessed image + normal reference	Calculate reference difference and threshold-based decision	Prediction, raw score, map and heatmap
`anomalib_trainer.py`	Normal training images / checkpoint	Train or load PaDiM and run anomaly inference	Anomaly score, map and predicted mask
`defect_classifier.py`	Image/features	Extract ResNet18 features, scale and classify	Defect type and probabilities
`heatmap.py`	Anomaly map + image	Normalize/colorize anomaly maps, create overlays and calculate localization overlap	Heatmap path / image
`severity.py`	Area, location, type and confidence scores	Apply weighted rule-based policy	Severity score, level and action
`predict.py`	Validated uploaded image	Build configuration and call the backend-compatible inference orchestrator	Backend-ready dictionary/JSON
`metrics.py`	Labels and predictions	Calculate AUROC, F1, accuracy and confusion outputs	Evaluation records and plots

7. Repository, Notebooks and Model Artifacts

The project separates experimentation from reusable runtime logic. Notebooks document the learning and evaluation journey, while `ml/`, `models/` and `tests/` support integration and repeatability.

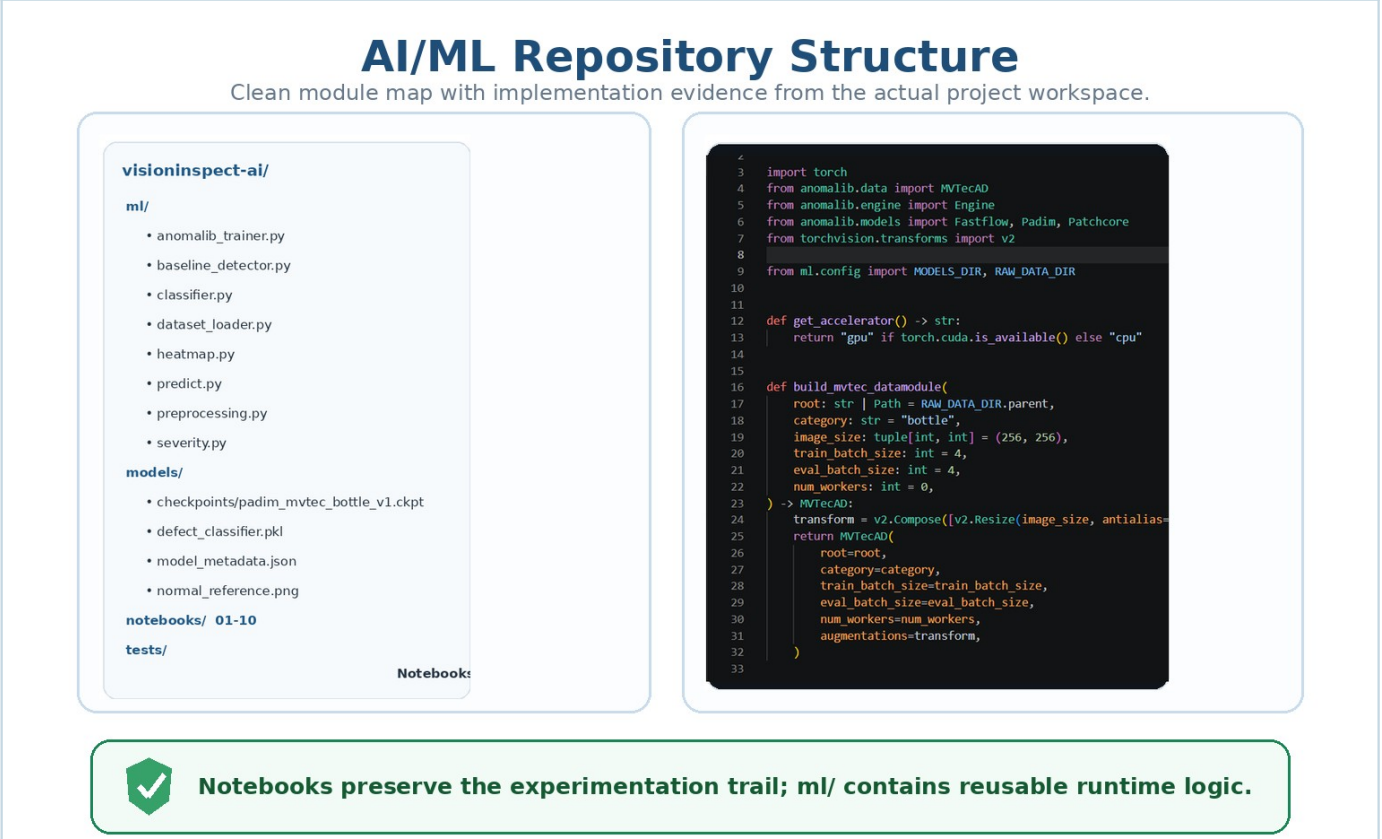


Figure 4. AI/ML repository map with reusable modules, artifacts, notebooks and implementation evidence.

What this proves: The module is organized beyond notebook-only experimentation.

Observation: The clean tree clarifies responsibility while the workspace image preserves authentic implementation evidence.

7.1 Notebook journey

Notebook group	Files	Purpose
Image foundations	01–03	Digital image basics, OpenCV/NumPy handling and preprocessing evidence
Data understanding	04–05	MVTec folders, classes, masks, counts and loader validation
Detection	06–07	OpenCV baseline development and PaDiM training/inference
Decision intelligence	08–09	Defect-type classification and severity/QA policy
Final evidence	10	Metric comparison, final inference output and testing proof

7.2 Runtime artifacts

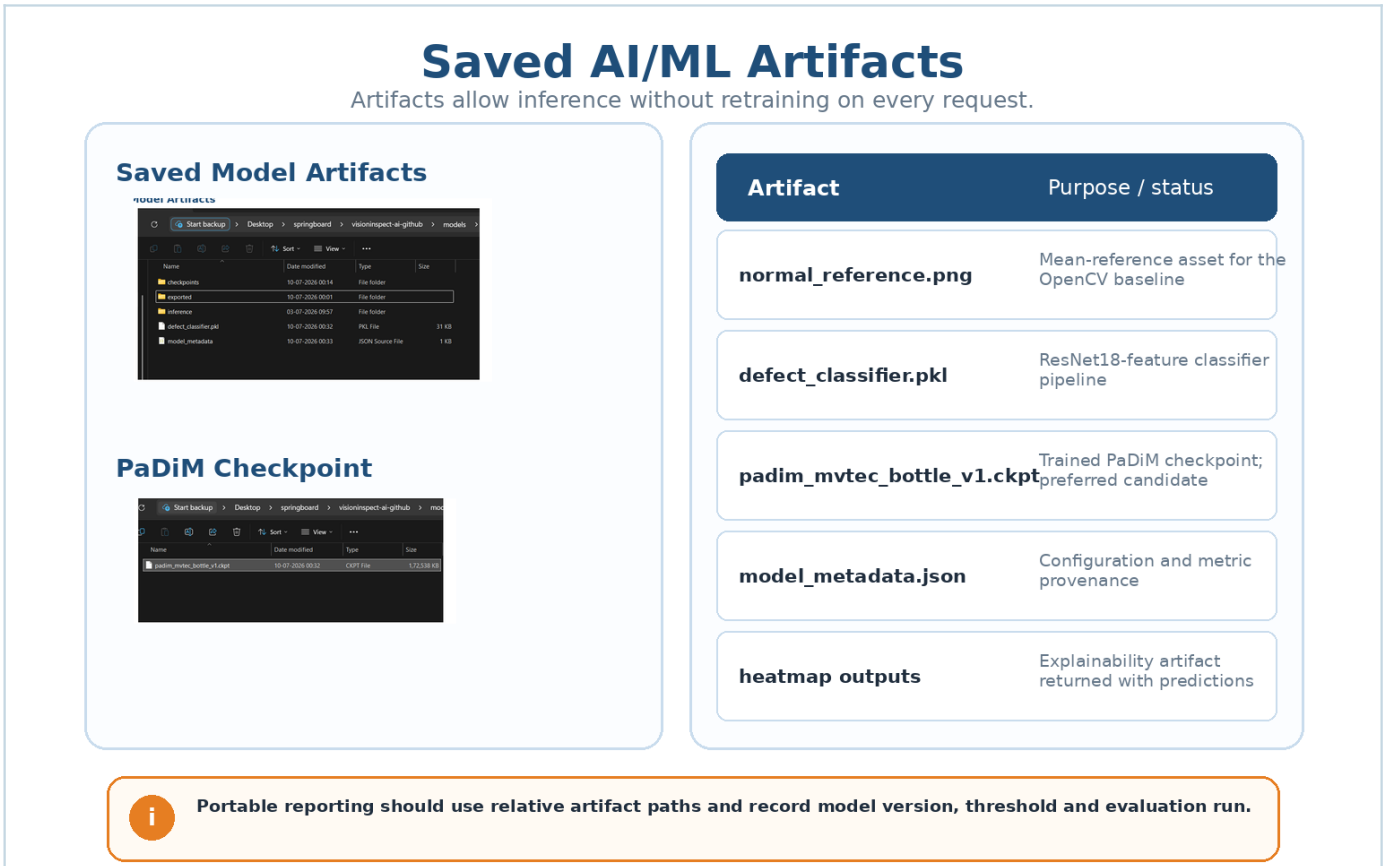


Figure 5. Saved AI/ML artifacts used by baseline, PaDiM, classification and explainability paths.

What this proves: Inference can load saved assets without retraining for every request.

Observation: Artifact metadata should use portable relative paths and record version, threshold and evaluation provenance.

Artifact	Used by	Purpose
`defect_classifier.pkl`	Classifier / prediction path	Stores the trained scaler + Logistic Regression classifier pipeline
`model_metadata.json`	Prediction and evaluation	Records model name/version, configuration and metric provenance
`normal_reference.png`	OpenCV baseline	Mean reference built from 209 normal training images
`padim_mvtec_bottle_v1.ckpt`	PaDiM inference	Stores trained PaDiM normal-pattern parameters
Heatmap outputs	Backend/frontend	Provides explainable localization evidence

8. Dataset, Splits and Validation

The project uses the MVTec Anomaly Detection Bottle category. Normal images are available for learning normal appearance, while test folders contain normal and defective cases. Ground-truth masks support pixel-level localization evaluation for defective samples.

Dataset partition	Count	Use in the project
`train/good`	209	Learn normal appearance and construct reference/model features
`test/good`	20	Measure false alarms on normal products
`test/broken_large`	20	Evaluate large structural breakage
`test/broken_small`	22	Evaluate smaller structural breakage
`test/contamination`	21	Evaluate contamination/foreign-material anomaly

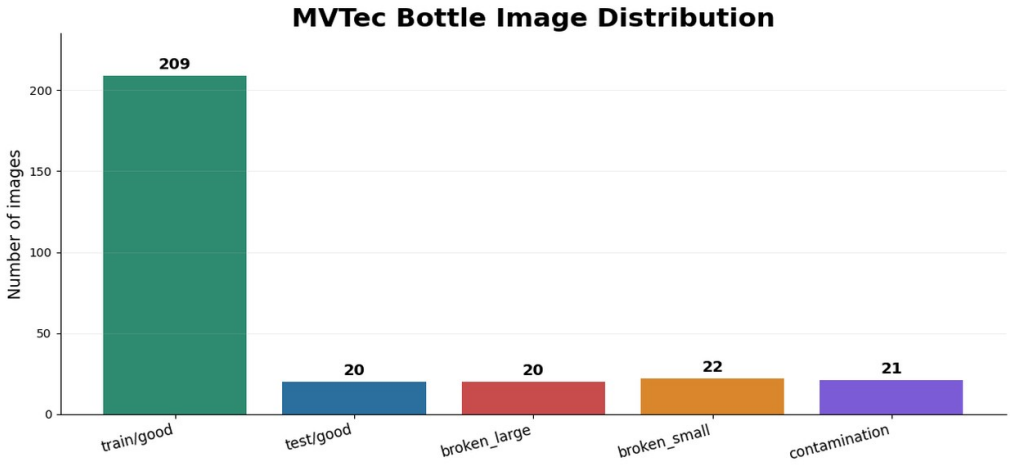


Figure 6. MVTec Bottle image distribution used for normal training and evaluation.

What this proves: The report records the actual class counts rather than only folder names.

Observation: PaDiM learns from 209 normal images; the 83 test images are comparatively small and class-imbalanced.

Figure 7. Real MVTec Bottle examples for good, broken_large, broken_small and contamination.

What this proves: All four classifier output classes are represented using real dataset images.

Observation: The breakage categories share fracture-like visual cues, which explains part of the observed confusion.

8.1 Dataset and sample validation

Validation check	Expected behaviour
Required folders	Confirm train, test and ground-truth structure exists
Image readability	Reject missing or unreadable image files
Class labels	Map folder names to consistent output labels
Ground-truth path	Associate defective image with mask where available
Dimensions and channels	Record height, width, channels and dtype before preprocessing
Split integrity	Keep normal training data separate from evaluation samples

9. Image Understanding and Preprocessing

9.1 Image as numerical input

A bottle image is represented as a numeric array rather than a human-readable picture. Before inference, the module checks whether the file can be decoded and records its dimensions, channel count and data type. Pixel values are then transformed into the representation expected by each model path.

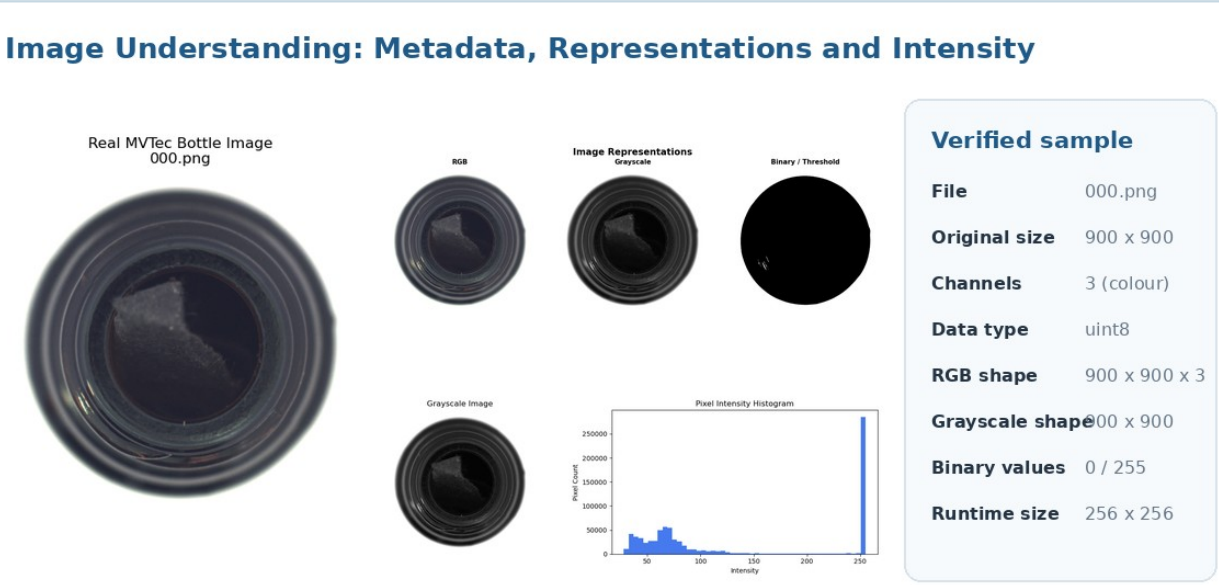


Figure 8. Loaded image metadata, RGB/grayscale/binary representations and pixel-intensity distribution.

What this proves: The image is inspected as a numerical array before model-specific transformation.

Observation: The verified sample is a three-channel uint8 image; runtime paths standardize spatial input to 256 x 256.

9.2 Common preprocessing versus path-specific preprocessing

Stage	What changes	Shared / path usage
Image reading	Decodes file into an image array	Required by all paths
Validation	Checks readability, shape, channel count and dtype	Required by all paths
BGR → RGB conversion	Corrects OpenCV channel order for display/model consistency	Shared when OpenCV reading is used
Resize to 256 × 256	Standardizes spatial input to 256 x 256	Shared confirmed input size
Tensor conversion / normalization	Converts RGB values to the numeric scale expected by pretrained deep features	PaDiM and ResNet18 classifier transforms
Grayscale conversion	Reduces RGB to intensity representation	Baseline and visual analysis
Denoising	Gaussian blur with a 5 x 5 kernel suppresses small pixel variation	Confirmed baseline-oriented processing
Contrast enhancement	CLAHE with clip limit 2.0 and 8 x 8 tile grid enhances local contrast	Confirmed baseline-oriented processing
Brightness / shadow correction	Explored to reduce illumination imbalance	Notebook/exploratory robustness evidence; not a direct PaDiM input
Thresholding	Converts image or anomaly response into binary cues/mask	Exploratory cue; runtime mask follows the active detector threshold
Edge detection	Canny thresholds 100 and 200 highlight structural discontinuities	Explainable cue; not a PaDiM input

9.3 Detailed Preprocessing Evidence

9.3.1 Step-by-step transformation

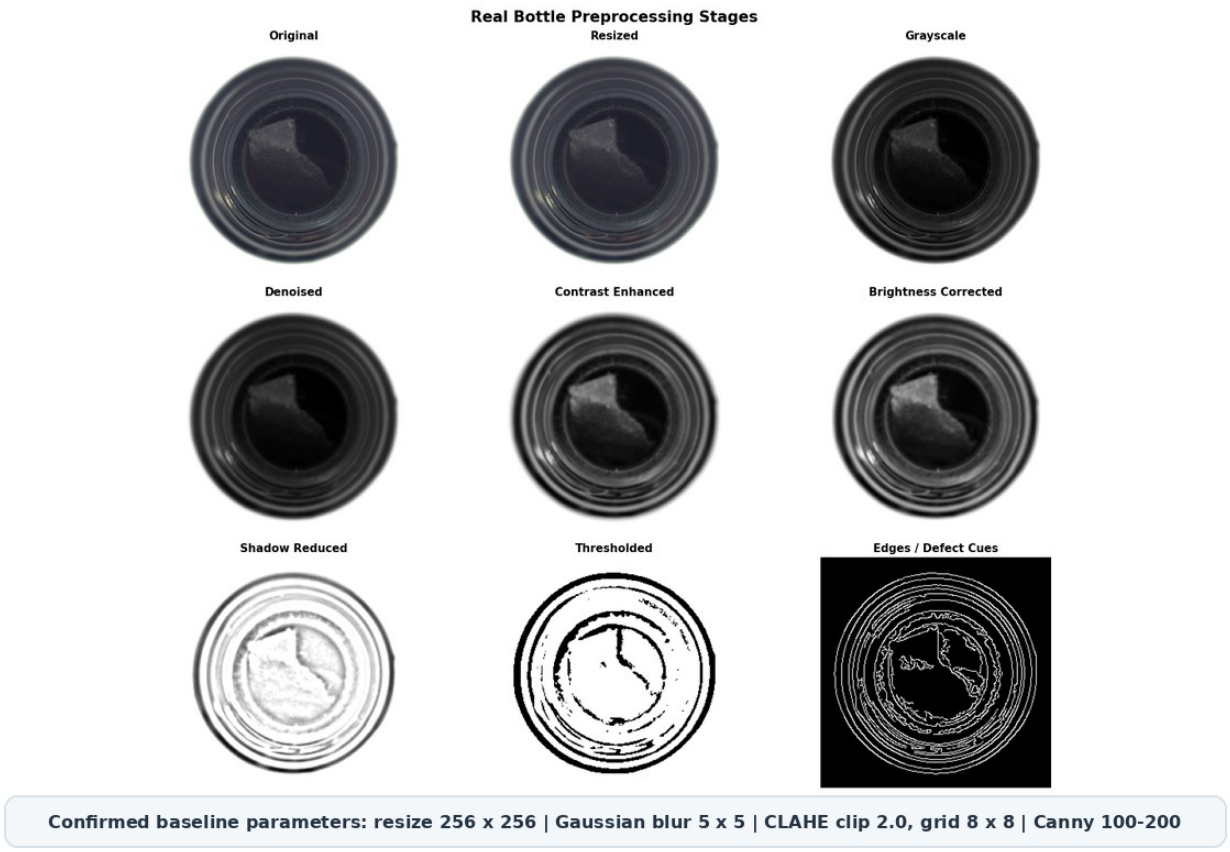


Figure 9. Real MVTec bottle preprocessing stages and explainable defect cues.

What this proves: The preprocessing description is grounded in a real dataset sample and confirmed parameters.
Observation: Resize, grayscale, Gaussian blur and CLAHE support the baseline; threshold and edge views are explainability cues.
Limitation: PaDiM and the classifier consume their model-specific resized RGB transforms rather than binary edge images.

9.4 Before-versus-after interpretation

Original / RGB Preserves full colour and surface appearance for deep feature extraction.	**Grayscale** Condenses colour information into intensity for simpler comparison.
Denoised Reduces small random variations that could create false difference responses.	**Enhanced** Improves visibility of subtle contamination or fracture boundaries.
Thresholded Produces coarse foreground/defect cues for explainable baseline logic.	**Edges** Highlights structural discontinuities but can also respond to normal bottle rings.

Confirmed preprocessing parameters
Input size: 256 x 256; Gaussian blur: 5 x 5; CLAHE: clip limit 2.0, tile grid 8 x 8; Canny thresholds: 100 and 200. Deep-model normalization follows the active PaDiM/ResNet18 transform configuration.

10. OpenCV Baseline Detector

The OpenCV baseline compares each preprocessed image with a mean normal reference constructed from all 209 train/good images. It applies the confirmed grayscale, Gaussian-blur and CLAHE pipeline, calculates an absolute-difference map and converts the resulting score into a Good/Defective decision using a configured threshold. The method is fast and explainable but sensitive to alignment, illumination and normal ring variation.

10.1 Input–process–output

Input	Processing	Output
Validated bottle image + mean `normal_reference.png` from 209 good images	Resize 256 x 256, grayscale, Gaussian blur 5 x 5, CLAHE 2.0/8 x 8, absolute difference and threshold decision	Good/Defective, raw difference score, binary mask, affected-image ratio, anomaly map and heatmap

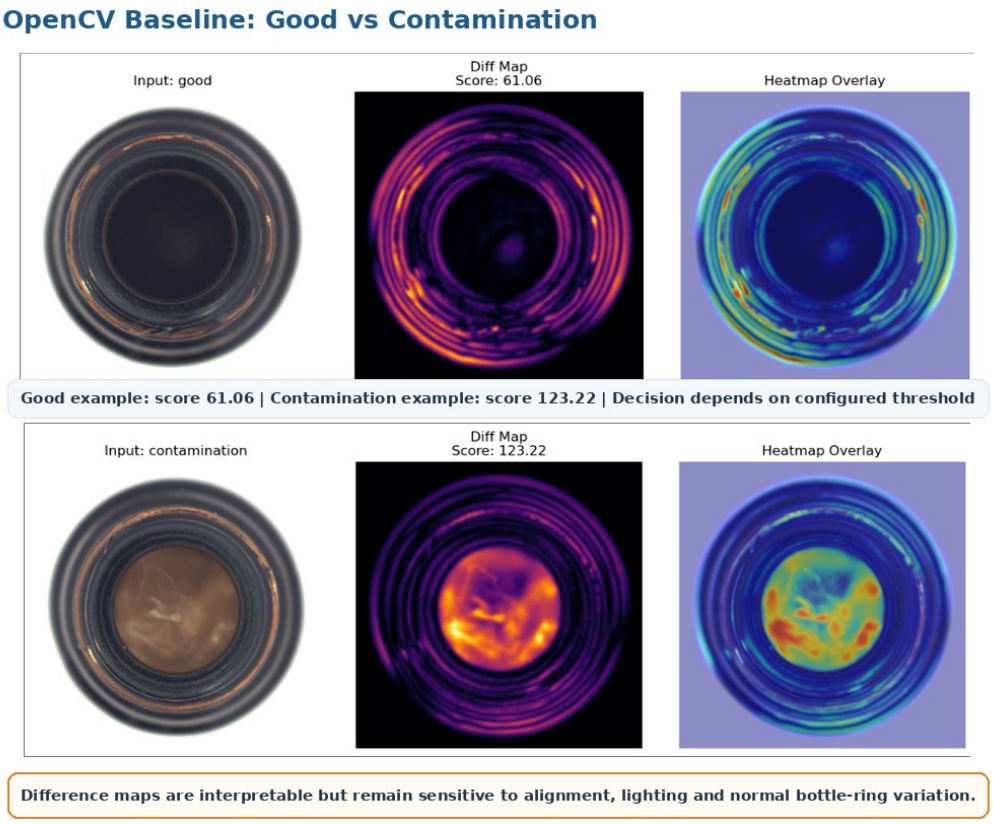


Figure 10. OpenCV baseline comparison for a normal and a contamination sample.

What this proves: The baseline produces interpretable difference maps and overlays for both low- and high-score examples.
Observation: The contamination sample has a substantially larger central response than the normal sample.
Limitation: Normal bottle rings can also produce high responses, so threshold selection and image alignment remain critical.

10.2 Threshold and score interpretation

The baseline anomaly score is a raw image-difference value. The reported accuracy 51.81% and defect recall 36.51% were calculated at the notebook 99th-percentile threshold of 78.4423. The supplied runtime configuration uses approximately 61.99, so the report does not treat the 99th-percentile metrics as direct measurements of the deployed operating point. ROC-AUC 92.06% describes ranking across thresholds.

Metric	Result	Interpretation
Detection accuracy	51.81%	Approximately half of evaluation cases are classified correctly at the selected operating point
Detection error rate	48.19%	1 - accuracy at the selected 78.4423 evaluation threshold
Defect recall	36.51%	Many defective images can be missed

Metric	Result	Interpretation
ROC-AUC	92.06%	Ranking ability is reasonable, but threshold calibration and input sensitivity remain poor

Deployment position
The supplied Render configuration uses the OpenCV baseline as a demonstration/fallback path. Its low recall at the reported 99th-percentile evaluation point means it must not be represented as a production-certified detector. Runtime threshold 61.99 requires its own final metric report.

11. PaDiM Anomaly Detection and Localization

PaDiM models the distribution of pretrained visual features at spatial locations using normal bottle images. The confirmed configuration uses a ResNet18 backbone, feature layers layer1, layer2 and layer3, 100 selected features and 256 x 256 input. During inference, the checkpoint produces an image-level anomaly score and a pixel-level anomaly map that is thresholded into a predicted mask and visualized as a heatmap.

11.1 Training and inference logic

Phase	Input	Operation	Output
Training	Normal bottle images	Resize to 256 x 256, extract ResNet18 layer1/layer2/layer3 features, retain 100 selected features and model normal distributions	PaDiM checkpoint and normal-pattern statistics
Inference	Validated test image + checkpoint	Apply the same transform, compare spatial features with the learned normal distributions and threshold image/pixel outputs	Image score, anomaly map, predicted mask and heatmap
Evaluation	Image labels + ground-truth masks	Compare image decisions and pixel localization	Image AUROC/F1 and Pixel AUROC/F1

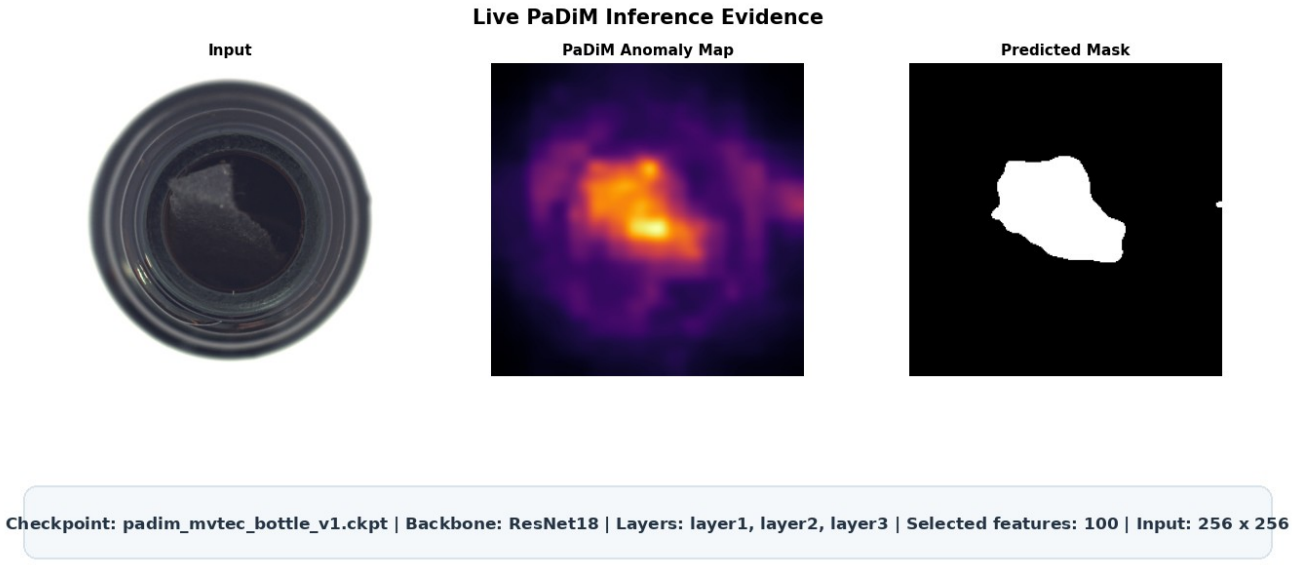


Figure 11. PaDiM checkpoint inference showing input image, anomaly map and predicted mask.

What this proves: The trained anomaly path provides both image-level evidence and spatial localization.

Observation: The strongest anomaly response and predicted mask align with the visible contamination region.

Limitation: The checkpoint is trained and evaluated but is disabled in the supplied Render configuration; activation and runtime threshold validation remain pending.

Metric	Result	Interpretation
Image AUROC	99.84%	Excellent separation of normal and defective images
Image F1	98.39%	Excellent image-level precision/recall balance
Pixel AUROC	98.21%	Strong ranking of abnormal versus normal pixels
Pixel localization F1	71.36%	Useful but moderate exact overlap with defect boundaries

Model selection conclusion

PaDiM is the stronger anomaly-detection candidate. Image AUROC 99.84% and Image F1 98.39% support screening on the evaluated MVTec setup, while Pixel F1 71.36% requires honest interpretation: localization is useful but not exact segmentation.

12. Defect-Type Classification

The defect classifier is invoked only when the anomaly detector marks an image as defective. A frozen pretrained ResNet18 converts the image into a visual embedding; StandardScaler standardizes the embedding; and balanced Logistic Regression (max_iter=3000, random_state=42) estimates good, broken_large, broken_small or contamination probabilities. If the classifier predicts good for an already detected anomaly, the runtime changes the category to unknown_defect.

12.1 Architecture

Step	Component	Role
1	ResNet18 feature extractor	Frozen pretrained network converts image content into visual embeddings
2	StandardScaler	Fitted only on training embeddings and reused during inference
3	Logistic Regression	Balanced multiclass classifier; max_iter=3000; random_state=42
4	Probability output	Returns per-class probabilities; highest value is a model confidence estimate

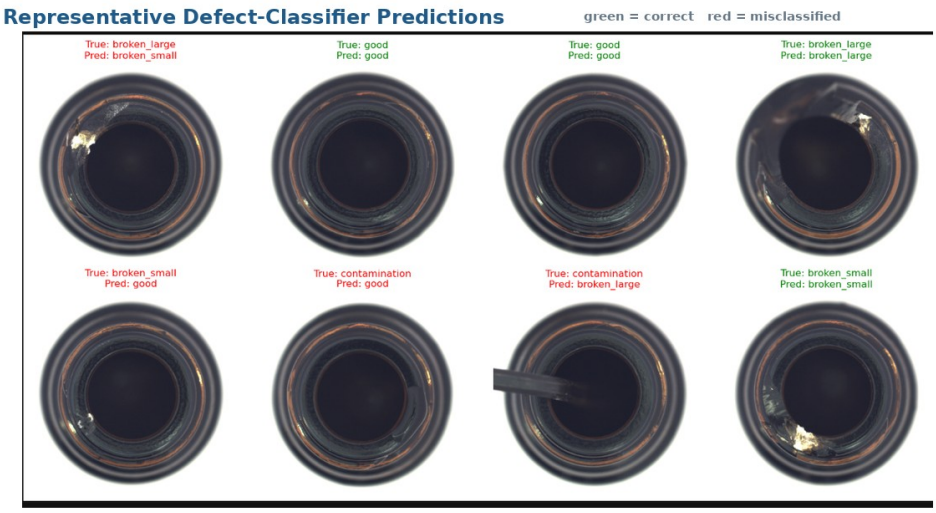


Figure 12. Representative correct and incorrect defect-classifier predictions.

What this proves: The classifier returns category-level predictions for real bottle samples.

Observation: Most correct examples are clear, while breakage classes and subtle contamination can be visually confusable.

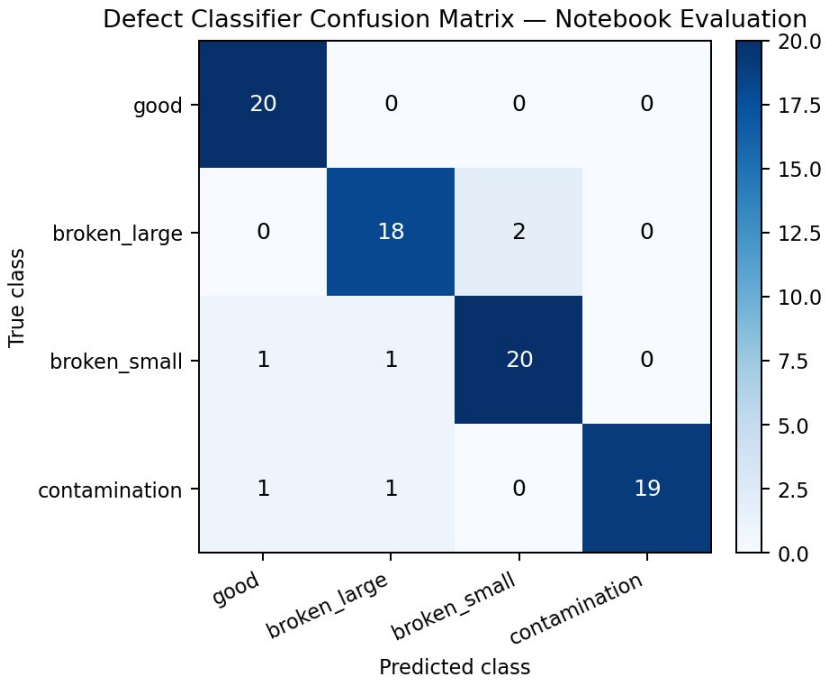


Figure 13. Confusion matrix for the verified 83-sample evaluation.

What this proves: The evaluation records class-by-class correct and incorrect predictions.

Observation: The matrix contains 77 correct predictions out of 83, giving 92.77% accuracy; the main confusion is between the two breakage classes.

Limitation: A separate 93.18% historical record is not used as the headline metric because its 88-sample split and artifact provenance are not present in the supplied evidence package.

12.2 Metric provenance and mismatch resolution

Metric source	Accuracy	Error	Report treatment
Verified confusion-matrix evaluation (83 samples)	92.77%	7.23%	Headline classifier result; directly supported by the displayed matrix
Separate historical record (reported as 88 samples)	93.18%	6.82%	Retained only as a historical record until split, predictions and artifact provenance are supplied

13. Explainability, Severity and QA Decision

13.1 Explainability outputs

Output	Generated by	What it explains
Anomaly score	Baseline or PaDiM	How strongly the image differs from normal appearance
Anomaly map	PaDiM / detector	Pixel-wise abnormality response
Heatmap overlay	Heatmap module	Where suspicious evidence lies on the original image
Predicted mask	Localization threshold	Which pixels are treated as defective
Defect-area ratio	Mask / localization	Fraction of pixels in the resized image marked anomalous by the predicted mask
Ground-truth comparison	Evaluation only	How predicted localization overlaps annotated defect region
Classifier confidence	Defect classifier	Model probability estimate supporting the selected category; not a calibrated reliability guarantee

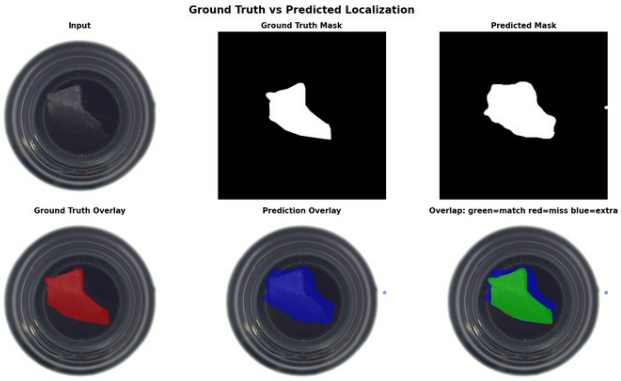


Figure 14. Ground-truth and predicted localization comparison for a representative contamination sample.

What this proves: Pixel-level evaluation compares the predicted mask with annotated defect regions rather than only image labels.

Observation: The example shows 7.25% ground-truth area, 9.70% predicted area, IoU 0.7328, Dice 0.8458 and 98.89% ground-truth coverage.

13.2 Severity is a rule-based decision layer

Severity scoring is a rule-based decision layer. It converts localization and classification evidence into four component scores: defect size 30%, defect location 25%, defect type 25% and confidence 20%. Size is mapped from affected-image ratio; location uses the critical centre-region heuristic; type uses a version-controlled risk table; and confidence is converted to a 0-100 score.

Component	Weight	Source
Defect size	30%	Area ratio bands: 0, 20, 45, 70 or 90
Defect location	25%	Critical=True -> 90; centre heuristic -> 65; otherwise 35
Defect type	25%	Example risk scores: contamination 55, broken_small 75, broken_large 95
Confidence	20%	Classifier/detection confidence converted to 0-100 and clipped

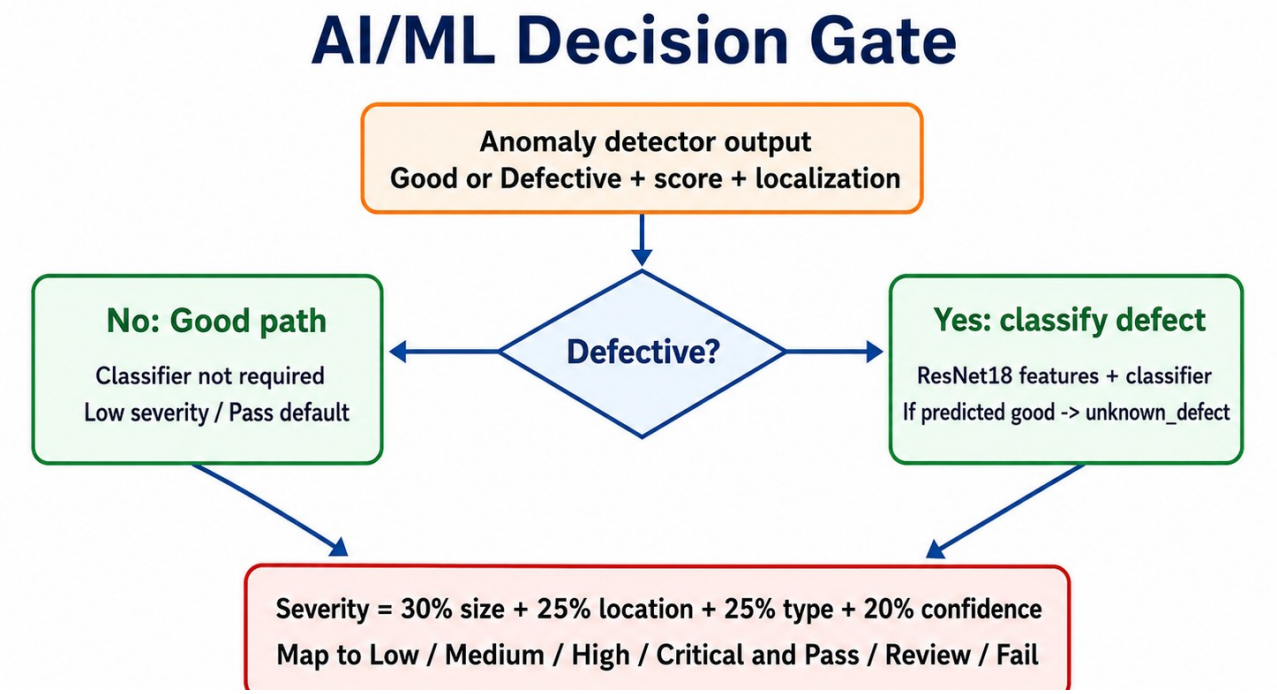


Figure 15. AI/ML decision gate with conditional classifier and disagreement handling.

What this proves: The detector controls whether defect classification is required before severity calculation.

Observation: Good cases bypass the classifier; a detected anomaly classified as good is relabelled unknown_defect before decision support.

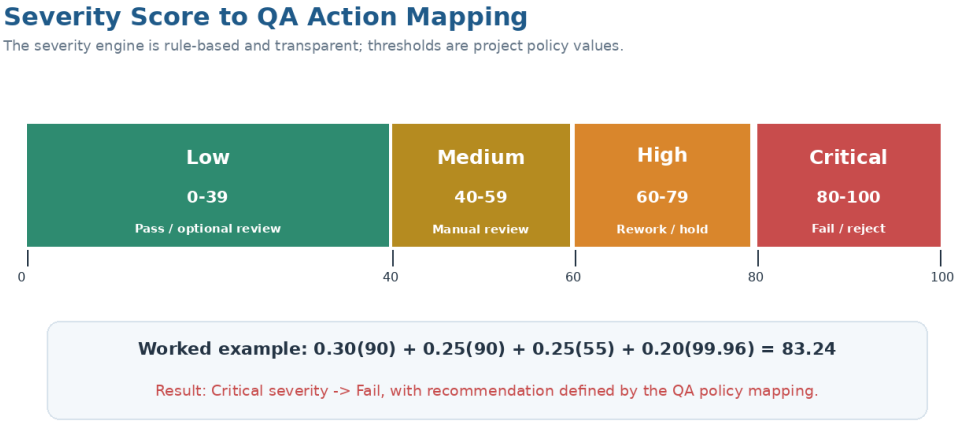


Figure 16. Severity score ranges, QA actions and the verified 83.24 worked example.

What this proves: The final score is converted into an operational action using transparent thresholds.

Observation: The example calculation is 0.30(90) + 0.25(90) + 0.25(55) + 0.20(99.96) = 83.24, resulting in Critical / Fail.

13.3 Real inference example

Field	Reported value
Prediction	Defective
Defect type	contamination
Confidence	99.96%
Anomaly score	0.679
Defect-area ratio	9.7%
Severity score	83.24
Severity level	Critical
QA decision	Fail
Size component	90 x 30% = 27.00
Location component	90 x 25% = 22.50
Defect-type component	55 x 25% = 13.75
Confidence component	**99.96 x 20% = 19.99**

14. Input/Output Contract and Invalid-Input Handling

14.1 Input contract

Contract item	Expected behaviour
Input	One uploaded product image or valid image path/stream payload supplied to the AI/ML boundary
Readability	Image must be decodable by the active image-reading library
Channels	Colour inputs are expected; supported grayscale/RGBA inputs may be converted only when validated
Spatial size	Runtime processing standardizes input to 256 × 256
Data type	Image array is validated before conversion to model-specific numeric representation
Traceability	Source filename, active engine, model version, threshold and fallback status should be carried into the result

14.2 Invalid-input behaviour

Invalid case	Required handling
Text/document/non-image file	Reject before model inference; platform validation returns a 400-level error
Corrupted or unreadable image	Stop processing and return an invalid-image failure response
Empty payload or missing path	Stop processing and return a missing/empty-input error
Unsupported channel format	Convert only when safe; otherwise reject
Unexpected dimensions	Resize only after successful decoding and validation
Model artifact unavailable	Return controlled inference-unavailable error or use the explicitly configured baseline fallback with a recorded reason

14.3 Output contract

Field	Source	Meaning / consumer	Required
prediction	Anomaly detector	Good/Defective decision for backend and result card	Yes
anomaly_score	Baseline or PaDiM	Method-specific abnormality value	Yes
heatmap_path	Heatmap/localization	Visual evidence for frontend display	Conditional
defect_area_ratio	Predicted mask	Predicted anomalous pixels divided by total resized-image pixels	Yes
defect_type	Classifier	Selected category for reporting and analytics	Conditional
confidence	Classifier probability	Model confidence estimate used by UI and severity rules	Conditional
severity_score	Rule engine	0–100 QA risk score	Yes
severity_level	Rule mapping	Low, Medium, High or Critical	Yes
pass_fail	QA policy	Pass, Review or Fail action	Yes
model_used	Runtime metadata	Model/version and active anomaly-engine traceability	Yes
active_inference_engine	Runtime metadata	Records baseline or PaDiM	Yes
fallback_used / fallback_reason	Runtime orchestration	Makes automatic PaDiM-to-baseline fallback auditable	Conditional
severity_components	Severity engine	Exposes size, location, type and confidence contributions	Yes

Score-scale warning

Baseline raw difference score, PaDiM anomaly score, classifier confidence and severity score are different quantities. The frontend and report must label each field clearly and must not compare the numeric values as if they share one scale.

15. AI/ML Output Contract and Integration Boundary

The final AI/ML orchestration stage assembles the active detector/localizer, conditional classifier and severity engine into one JSON-compatible result. The backend is responsible only for supplying a validated path/payload and consuming or storing the returned fields; the frontend is a display consumer. This section documents the integration boundary rather than claiming ownership of backend or frontend implementation.

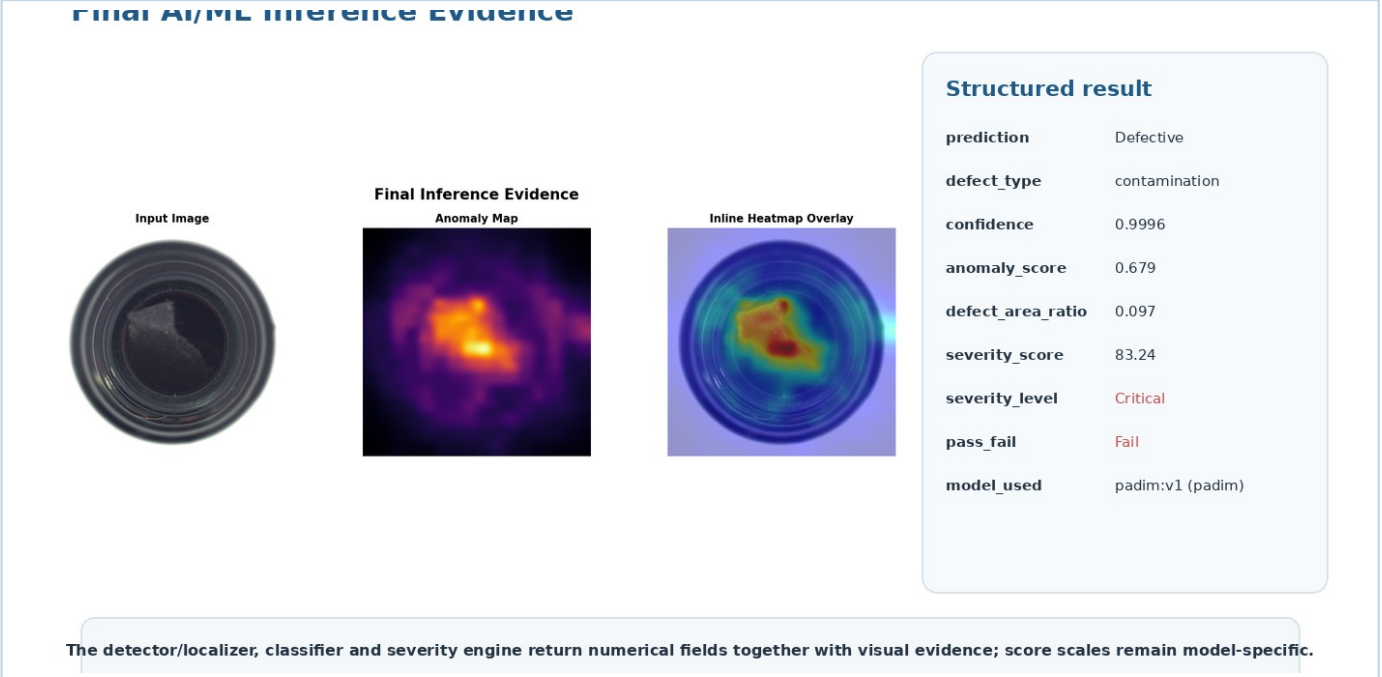


Figure 17. Final AI/ML inference evidence combining structured fields and visual localization.

What this proves: The module returns detector, classifier, localization and severity evidence together.

Observation: The example reports contamination, confidence 0.9996, anomaly score 0.679, affected-image ratio 0.097, severity 83.24 and Fail.

Limitation: The image is notebook/checkpoint evidence; live deployment must report its actual active engine and fallback state.

15.1 Field-to-platform mapping

AI/ML field	Integration-boundary use	Example display use
Prediction / decision	Consume inspection status; trigger workflow	Pass/Review/Fail card
Defect type	Consume category for analytics/reports	Defect-type label
Confidence	Consume model evidence	Confidence percentage
Anomaly score	Consume method-specific score	Explainability detail
Defect-area ratio	Consume localization summary	Affected-area detail
Severity score / level	Drive QA routing and reporting	Severity badge and recommended action
Heatmap path/image	Consume/serve visual artifact	Defect heatmap panel
Model used	Audit model version	Optional traceability detail

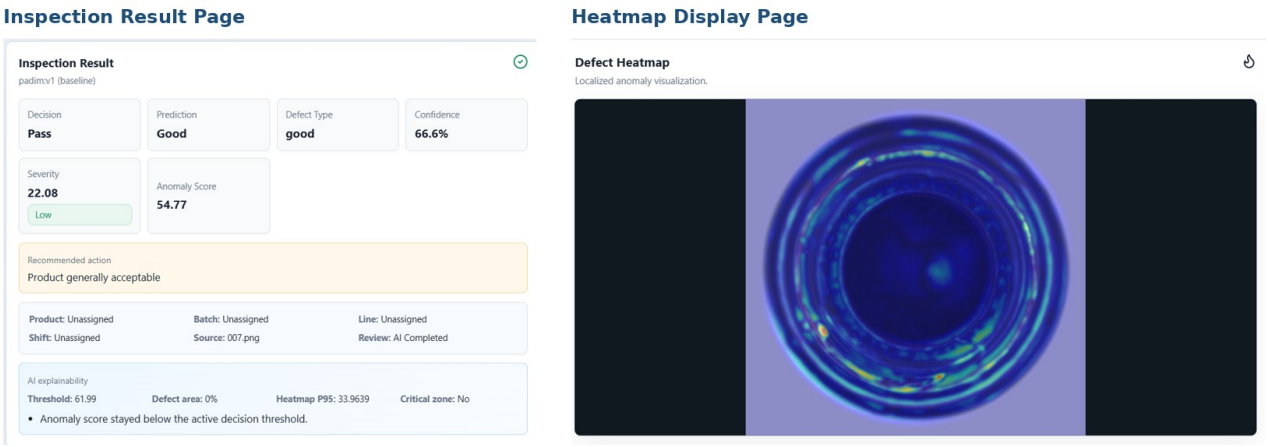


Figure 18. Platform display evidence consuming AI/ML fields and heatmap output.

What this proves: The integration demonstration confirms that numerical fields and localization evidence can be consumed outside the AI/ML module.

Observation: The figure is supporting integration evidence, not a claim of frontend implementation ownership.

16. Model Evaluation and Production Readiness

16.1 Consolidated metric view

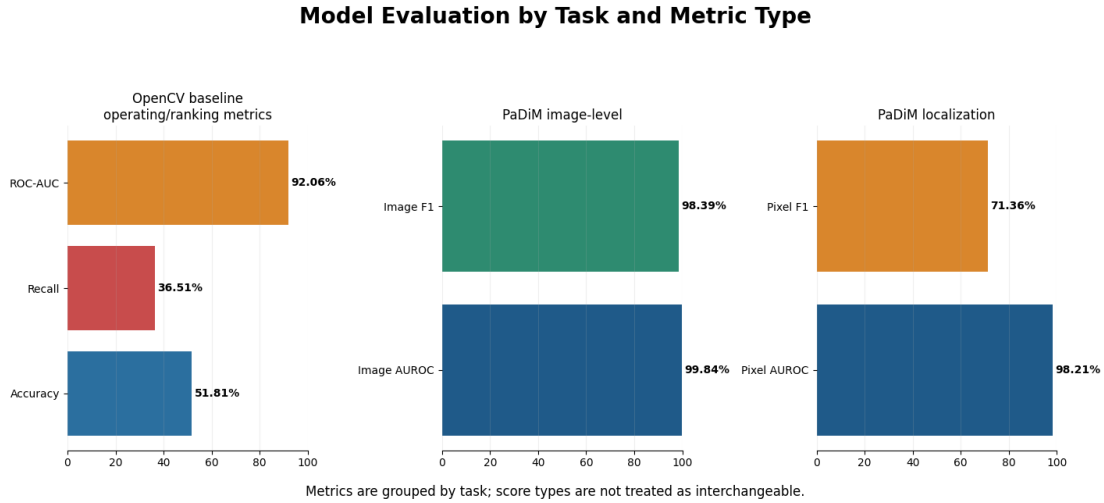


Figure 19. Model evaluation grouped by task and metric type.

What this proves: Metrics are grouped by operating-point detection, image-level anomaly detection and pixel localization.

Observation: PaDiM is substantially stronger at image-level detection; baseline recall is the main demonstrated detection risk.

Limitation: Accuracy, AUROC, F1 and pixel overlap answer different questions and must not be interpreted as one interchangeable scale.

System component	Metric	Result	Decision relevance
OpenCV baseline	Detection accuracy	51.81%	Insufficient for professional production screening
OpenCV baseline	Detection error rate	48.19%	High error risk at current operating point
OpenCV baseline	Defect recall	36.51%	False negatives are a critical concern
OpenCV baseline	ROC-AUC	92.06%	Ranking is useful; threshold/input robustness remain weak
PaDiM	Image AUROC	99.84%	Strong anomaly separation
PaDiM	Image F1	98.39%	Strong image-level operating performance
PaDiM	Pixel AUROC	98.21%	Strong anomaly-pixel ranking
PaDiM	Pixel F1	71.36%	Exact boundary overlap remains an improvement area

System component	Metric	Result	Decision relevance
Classifier	Verified classifier accuracy	92.77%	77/83 correct in the directly supported confusion-matrix evaluation
Classifier	Verified classifier error	7.23%	Historical 93.18% record excluded from headline comparison pending provenance

16.2 Production-readiness interpretation

Detection risk

The deployed baseline can miss defects; false-negative monitoring and PaDiM activation are priorities.

Localization quality

PaDiM localizes meaningful regions, but pixel F1 shows boundaries are not exact.

Classification quality

Category prediction is strong, but breakage classes remain visually confusable.

Policy validation

Severity thresholds and critical zones require manufacturing-domain approval.

Environment robustness

Lighting, background, orientation and camera position require real-world validation.

Compute and latency

PaDiM deployment must be tested against available CPU/GPU and response-time targets.

16.3 Model comparison summary

Method	Primary strength	Primary limitation	Recommended role
OpenCV baseline	Fast and interpretable	Low recall; sensitive to imaging conditions	Demo/fallback and sanity check
PaDiM	Excellent image anomaly detection and useful localization	Deployment and exact mask calibration pending	Preferred anomaly-detector candidate
Defect classifier	Strong category probabilities	Small labelled set; similar breakage classes; confidence not formally calibrated	Defect-type evidence path
Severity engine	Transparent QA mapping	Rules require domain calibration	Decision support layer

17. Automated Testing and Reliability

Automated tests validate selected software behaviours and are kept separate from predictive model evaluation. The recorded submission screenshot reports 19 tests passed in 31.93 seconds. AI/ML-relevant tests cover dataset loading, preprocessing, baseline logic, classifier artifacts, heatmaps, validation utilities, prediction output and severity; API/authentication tests are treated only as integration evidence.

Test area	Assurance category / purpose
`test_api.py`	Platform integration evidence; not counted as predictive model evaluation
`test_auth.py`	Platform integration evidence; not counted as predictive model evaluation
`test_baseline_detector.py`	Baseline scoring and decision functions (AI/ML software behaviour)
`test_camera_samples.py`	Platform integration evidence; not counted as predictive model evaluation
`test_classifier_artifact.py`	Classifier artifact presence and loading (AI/ML software behaviour)
`test_dataset_loader.py`	Dataset records and paths (AI/ML software behaviour)
`test_kfold_validation.py`	Validation utility behaviour (AI/ML software behaviour)
`test_prediction.py`	Final prediction response fields (AI/ML software behaviour)
`test_preprocessing.py`	Image transformation functions (AI/ML software behaviour)
`test_severity.py`	Severity score and decision mapping (AI/ML software behaviour)

Recorded Software Validation Run — 19 tests passed in 31.93 seconds (submission evidence snapshot)

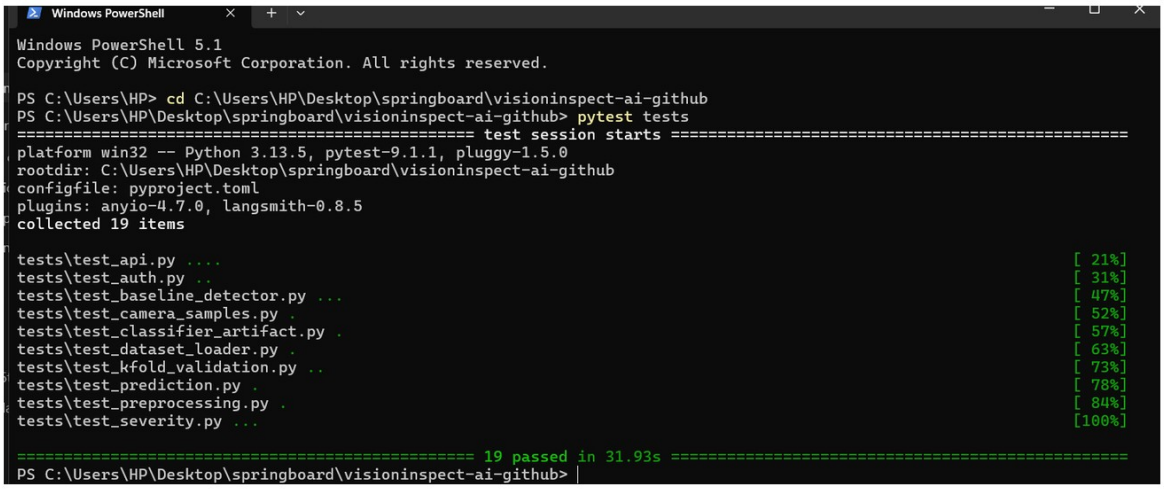


Figure 20. Recorded validation run showing 19 tests passed in 31.93 seconds.

What this proves: The selected software paths completed successfully in the recorded submission environment.

Observation: The run includes AI/ML module tests and integration-related checks; the report distinguishes them from model metrics.

Limitation: Passing tests does not prove predictive accuracy, threshold quality, latency or real-factory robustness.

Reliability distinction

Software testing, model evaluation and production validation are three different assurance layers. The report keeps them separate to avoid overstating readiness.

18. Challenges and Limitations

18.1 Implementation challenges

Dataset semantics Understanding normal-only training, defect folders and mask alignment.	**Real evidence** Replacing demonstration images with actual MVTec bottle samples.
Localization clarity Generating overlays that are visually meaningful and comparable with masks.	**Model integration** Connecting notebook-developed logic to reusable modules and response fields.
Artifact management Keeping classifier, checkpoint, metadata and reference files consistent.	**Metric consistency** Separating model runs, score scales and deployment status.

18.2 Current limitations

Limitation	Effect on interpretation or deployment
Bottle category only	Generalization to other products is not established
Baseline active on Render	Current live detector performance is weaker than trained PaDiM
Small defective class counts	The verified 83-sample classifier result can vary with split and class balance
Lighting/background sensitivity	Baseline difference maps can respond to non-defect variation
Camera angle/orientation sensitivity	Reference alignment and appearance can change
Pixel F1 71.36%	Exact defect boundary segmentation remains imperfect
Compute dependency	PaDiM latency and memory must be validated on deployment hardware
Rule-based severity	Weights, score bands and critical-zone heuristic require QA/domain approval
No full factory acceptance test	Offline metrics cannot be treated as production certification
Affected-image denominator	Defect-area ratio currently uses the entire resized mask/image, not a bottle-only ROI
Classifier provenance	The historical 93.18% record cannot be promoted without its split and artifact evidence

18.3 False-negative risk

Quality risk

The baseline defect recall of 36.51% means defective items can be missed. For manufacturing use, false-negative rate and recall must be tracked at the chosen threshold, and the deployed detector should be upgraded and validated before operational reliance.

19. Future Scope and Production Roadmap

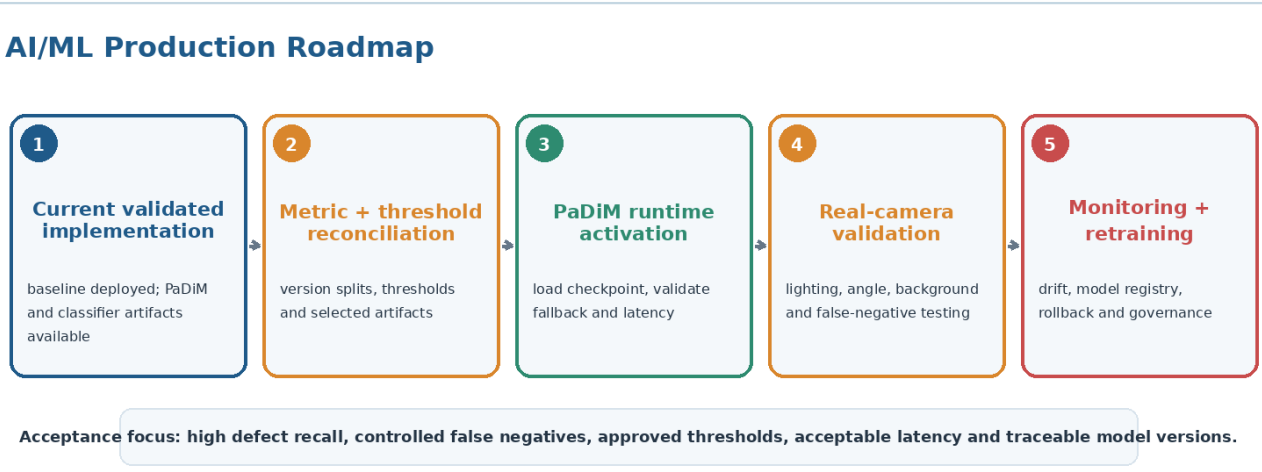


Figure 21. AI/ML roadmap from validated implementation to governed production use.

What this proves: Future work is prioritized around model risk, threshold traceability, real-image validation and monitoring.

Priority	Upgrade	Expected outcome
1	Activate and version PaDiM in the live anomaly path	Replace weak baseline detection with stronger trained anomaly model
2	Version model artifacts, dataset split and thresholds in metadata	Resolve metric ambiguity and improve traceability
3	Calibrate image and pixel thresholds using a separated validation procedure	Improve recall/F1 at the selected operating point
4	Collect factory-specific images across lighting, angle and background	Measure domain shift and real false-negative risk
5	Benchmark CPU/GPU latency, memory and throughput	Confirm deployment feasibility
6	Improve classifier validation and preserve split integrity	Reduce `broken_large` / `broken_small` confusion
7	Add product-ROI area calculation and tune mask post-processing	Improve pixel F1 and severity reliability
8	Add monitoring, drift detection and retraining governance	Support long-term operational reliability
9	Extend to additional product categories	Broaden platform use beyond bottle inspection

Production acceptance criteria to define

• Minimum defect recall and maximum false-negative rate by defect class.	• Approved anomaly and mask thresholds for each camera configuration.
• Maximum inference latency and hardware resource limits.	• Expected heatmap/localization quality for QA review.
• Severity and Pass/Review/Fail policy approved by manufacturing stakeholders.	• Model-version rollback and monitoring process.

20. Conclusion and Final Outcome

The VisionInspect AI module has been implemented as an end-to-end AI/ML inspection layer: image validation, path-specific preprocessing, configurable anomaly detection, localization, conditional defect classification, transparent severity scoring and JSON-compatible output are all represented in reusable code and evidence notebooks.

The implementation is supported by ten progressive notebooks, reusable Python modules, a mean baseline reference, a trained PaDiM checkpoint, a trained classifier artifact, real anomaly/mask visualizations and recorded software tests. The verified results show that PaDiM is the stronger detector candidate, while the deployed baseline remains a demonstration/fallback path requiring operating-point validation.

20.1 What has been achieved

Area	Final outcome
Data and input handling	MVTec Bottle structure, class counts, masks and invalid-input behaviour documented
Preprocessing	Shared and path-specific transformations explained with real-image evidence
Anomaly detection	Baseline implemented and PaDiM trained/evaluated
Classification	Defect category and confidence produced through a feature-based classifier
Explainability	Anomaly map, heatmap, predicted mask, area and ground-truth comparison supported
Decision support	Rule-based severity and QA action mapping implemented with a reproducible 83.24 worked example
Integration boundary	Structured output and supporting platform-consumption evidence demonstrated
Reliability	Metrics are separated by task; verified classifier accuracy is 92.77%; recorded 19-test run is documented

20.2 What remains before production reliance

Required step	Reason
Deploy and version PaDiM in the live backend	Current Render baseline recall is inadequate
Confirm and version classifier evaluation provenance	Use 92.77% as the verified result; promote 93.18% only after its split and artifact are supplied
Calibrate thresholds and severity policy	Current operating point and rules require validation
Run real-camera/factory testing	Offline MVTec performance does not guarantee environmental robustness
Monitor latency, drift and false negatives	Operational performance must be continuously measured

Final outcome

The assigned AI/ML module is a complete, explainable and test-supported implementation foundation. It is ready for the next engineering stage - PaDiM activation, threshold reconciliation, real-camera validation and governed monitoring - but is not yet factory-certified for autonomous quality decisions.

20.3 Reproducibility and Key References

Reproducibility item	Recorded value
Dataset	MVTec AD Bottle; train/good=209; test=20 good, 20 broken_large, 22 broken_small, 21 contamination
Baseline	Mean of 209 normal images; 256 x 256; Gaussian 5 x 5; CLAHE 2.0/8 x 8; evaluation threshold 78.4423
PaDiM	ResNet18; layer1/layer2/layer3; 100 selected features; padim_mvtec_bottle_v1.ckpt
Classifier	Frozen ResNet18 features; StandardScaler; balanced Logistic Regression; max_iter=3000; random_state=42
Verified headline results	PaDiM: 99.84% Image AUROC, 98.39% Image F1, 98.21% Pixel AUROC, 71.36% Pixel F1; classifier: 92.77%

[1] P. Bergmann et al., "MVTec AD - A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection," CVPR, 2019.
[2] T. Defard et al., "PaDiM: a Patch Distribution Modeling Framework for Anomaly Detection and Localization," ICPR Workshops, 2021.
[3] K. He et al., "Deep Residual Learning for Image Recognition," CVPR, 2016.
[4] OpenVINO/Anomalib project documentation for industrial anomaly detection workflows.
[5] OpenCV, PyTorch/torchvision and scikit-learn documentation for image processing, pretrained features and Logistic Regression.